HDOC = 41

NAME = ARAYANA KHARE

BATCH 15

SAP ID = 590042432

1. Aim

To write a menu-driven C program to read two paragraphs and perform the following operations using separate user-defined functions:

1. Find the longest and shortest words in each paragraph.

2. Remove repeated consecutive spaces.

3. Convert each paragraph into title case.

4. Find the frequency of every distinct word without considering letter case.

5. Display the results and exit the program.

2. Algorithm

1. Start the program.

2. Declare character arrays for two paragraphs and temporary strings.

3. Read both paragraphs from the user using `fgets()`.

4. Display the menu containing six options.

5. Read the user's choice.

6. If the choice is from 1 to 4, ask the user to select paragraph 1 or paragraph 2.

7. Use separate functions to perform the selected operation.

8. Display the result of the operation.

9. Repeat the menu until the user selects option 6.

10. Stop the program.

3. Complete C Program

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define SIZE 1000
#define WORD 100

void readParagraph(char paragraph[], size_t size);
void findLongestAndShortestWords(const char paragraph[],
                                 char longestWord[],
                                 char shortestWord[]);
void removeRepeatedSpaces(const char paragraph[],
                          char modifiedParagraph[]);
void convertToTitleCase(const char paragraph[],
                        char titleCaseParagraph[]);
void findWordFrequencies(const char paragraph[]);

int main()
{
```

```
char p1[SIZE], p2[SIZE];
char longest[WORD], shortest[WORD];
char modified[SIZE], title[SIZE];
int choice, para;

printf("Enter first paragraph:\n");
readParagraph(p1, SIZE);

printf("Enter second paragraph:\n");
readParagraph(p2, SIZE);

do
{
    printf("\n===== MENU =====\n");
    printf("1. Find longest and shortest words\n");
    printf("2. Remove repeated spaces\n");
    printf("3. Convert to title case\n");
    printf("4. Find word frequencies\n");
    printf("5. Display both paragraphs\n");
    printf("6. Exit\n");
    printf("Enter your choice: ");

    scanf("%d", &choice);
    getchar();

    if (choice >= 1 && choice <= 4)
    {
        printf("Choose paragraph (1 or 2): ");
        scanf("%d", &para);
        getchar();

        if (para != 1 && para != 2)
        {
            printf("Invalid paragraph number!\n");
            continue;
        }

        char *p = (para == 1) ? p1 : p2;

        switch (choice)
        {
            case 1:
                findLongestAndShortestWords(
                    p, longest, shortest);
```

```
                    printf("Longest word: %s\n", longest);
                    printf("Shortest word: %s\n", shortest);
                    break;

                case 2:
                    removeRepeatedSpaces(p, modified);
                    printf("Modified paragraph:\n%s\n",
modified);
                    strcpy(p, modified);
                    break;

                case 3:
                    convertToTitleCase(p, title);
                    printf("Title case paragraph:\n%s\n",
title);
                    strcpy(p, title);
                    break;

                case 4:
                    findWordFrequencies(p);
                    break;
            }
        }
        else if (choice == 5)
        {
            printf("Paragraph 1: %s\n", p1);
            printf("Paragraph 2: %s\n", p2);
        }
        else if (choice == 6)
        {
            printf("Program exited.\n");
        }
        else
        {
            printf("Invalid choice!\n");
        }

    } while (choice != 6);

    return 0;
}

void readParagraph(char paragraph[], size_t size)
{
    fgets(paragraph, size, stdin);
```

```
    paragraph[strcspn(paragraph, "\n")] = '\0';
}

void findLongestAndShortestWords(const char paragraph[],
                                 char longestWord[],
                                 char shortestWord[])
{
    char temp[SIZE];
    char *word;
    int first = 1;

    strcpy(temp, paragraph);
    word = strtok(temp, " \t\n");

    while (word != NULL)
    {
        int len = strlen(word);

        if (first)
        {
            strcpy(longestWord, word);
            strcpy(shortestWord, word);
            first = 0;
        }
        else
        {
            if (len > strlen(longestWord))
                strcpy(longestWord, word);

            if (len < strlen(shortestWord))
                strcpy(shortestWord, word);
        }

        word = strtok(NULL, " \t\n");
    }

    if (first)
    {
        strcpy(longestWord, "None");
        strcpy(shortestWord, "None");
    }
}

void removeRepeatedSpaces(const char paragraph[],
                          char modifiedParagraph[])
```

```
{
    int i = 0, j = 0;

    while (paragraph[i] != '\0')
    {
        if (paragraph[i] != ' ' || j == 0 ||
            modifiedParagraph[j - 1] != ' ')
        {
            modifiedParagraph[j++] = paragraph[i];
        }
        i++;
    }

    modifiedParagraph[j] = '\0';
}

void convertToTitleCase(const char paragraph[],
                        char titleCaseParagraph[])
{
    int i, newWord = 1;

    for (i = 0; paragraph[i] != '\0'; i++)
    {
        if (isspace((unsigned char)paragraph[i]))
        {
            titleCaseParagraph[i] = paragraph[i];
            newWord = 1;
        }
        else if (isalpha((unsigned char)paragraph[i]))
        {
            if (newWord)
                titleCaseParagraph[i] =
                    toupper((unsigned char)paragraph[i]);
            else
                titleCaseParagraph[i] =
                    tolower((unsigned char)paragraph[i]);

            newWord = 0;
        }
        else
        {
            titleCaseParagraph[i] = paragraph[i];
            newWord = 0;
        }
    }
```

```
    titleCaseParagraph[i] = '\0';
}

void findWordFrequencies(const char paragraph[])
{
    char temp[SIZE];
    char words[SIZE][WORD];
    char *token;
    int count[SIZE], n = 0, i, j, found;

    strcpy(temp, paragraph);

    for (i = 0; temp[i] != '\0'; i++)
        temp[i] = tolower((unsigned char)temp[i]);

    token = strtok(temp, " \t\n");

    while (token != NULL && n < SIZE)
    {
        found = 0;

        for (i = 0; i < n; i++)
        {
            if (strcmp(words[i], token) == 0)
            {
                count[i]++;
                found = 1;
                break;
            }
        }

        if (!found && strlen(token) < WORD)
        {
            strcpy(words[n], token);
            count[n] = 1;
            n++;
        }

        token = strtok(NULL, " \t\n");
    }

    printf("Word frequencies:\n");

    for (j = 0; j < n; j++)
```

```
        printf("%s : %d\n", words[j], count[j]);
}
```

4. Explanation of the Program

Step 1: Header files

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
```

- `stdio.h` provides `printf()`, `scanf()` and `fgets()`.

- `string.h` provides `strlen()`, `strcpy()`, `strcmp()` and `strtok()`.

- `ctype.h` provides `isspace()`, `isalpha()`, `toupper()` and `tolower()`.

Step 2: Read the paragraphs

```
void readParagraph(char paragraph[], size_t size)
{
    fgets(paragraph, size, stdin);
    paragraph[strcspn(paragraph, "\n")] = '\0';
}
```

- `fgets()` reads a complete line, including spaces.

- `strcspn()` locates the newline character.

- `'\0'` terminates the string at that position.

Step 3: Find the longest and shortest words

```
word = strtok(temp, " \t\n");
```
`strtok()` separates the paragraph into individual words.

```
int len = strlen(word);
```
`strlen()` calculates the length of the current word.

The program compares each word's length with the longest and shortest words found so far. It uses `strcpy()` to save the result in the output arrays.

Step 4: Remove repeated spaces

```
if (paragraph[i] != ' ' || j == 0 ||
    modifiedParagraph[j - 1] != ' ')
```

This condition copies a character unless it is an extra consecutive space. The variable `i` scans the original paragraph, while `j` tracks the position in the modified paragraph.

Step 5: Convert to title case

```
toupper((unsigned char)paragraph[i]);
tolower((unsigned char)paragraph[i]);
```

The first alphabetic character of each word is converted to uppercase, and the remaining alphabetic characters are converted to lowercase.

For example:

`hello WORLD programming` becomes `Hello World Programming`.

Step 6: Find word frequencies

First, the program converts the paragraph to lowercase. It then extracts each word using `strtok()` and compares it with previously stored words using `strcmp()`.

- If the word already exists, its count increases.

- If it is new, the program stores it and initializes its count to 1.

This makes `Hello`, `HELLO` and `hello` count as the same word.

5. Sample Output

```
Enter first paragraph:
hello world hello programming
Enter second paragraph:
C programming is easy

===== MENU =====
1. Find longest and shortest words
2. Remove repeated spaces
3. Convert to title case
4. Find word frequencies
```

```
5. Display both paragraphs
6. Exit
Enter your choice: 1
Choose paragraph (1 or 2): 1
Longest word: programming
Shortest word: hello

Enter your choice: 4
Choose paragraph (1 or 2): 1
Word frequencies:
hello : 2
world : 1
programming : 1

Enter your choice: 6
Program exited.
```

6. Result

The menu-driven C program successfully reads two paragraphs and performs string operations using separate user-defined functions, character arrays, pointers and standard string-handling functions.